4

Дополнительные сведения о пакетах

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Рассматриваемые вопросы

- Перезагрузка процедур и функций
- Использование предварительного объявления
- Создание блока инициализации в теле пакета
- Поддержка постоянного состояния пакетных данных в течение сеанса
- Использование в пакетах таблиц и записей PL/SQL
- Хранение в словаре данных исходного кода в преобразованном нечитаемом виде

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Цели урока

Этот урок описывает такие особенности PL/SQL, как *перезагрузка* (*overloading*), *предварительное объявление идентификаторов* (*forward referencing*), *процедура, выполняемая только один раз* (*one-time-only procedure*), а также устойчивое состояние переменных, констант, исключений и курсоров. Кроме того, рассматриваются эффекты использования пакетных функций в командах SQL.

Перезагрузка подпрограмм

- **Возможность создания нескольких подпрограмм с одним и тем же именем**
- **Формальные параметры подпрограмм должны отличаться по количеству, порядку или семейству типов данных**
- **Гибкий метод вызова подпрограммы с различными данными**
- **Расширение функциональных возможностей с сохранением существующего кода**

Примечание: только локальные, пакетные подпрограммы и методы типов могут быть перезагружаемыми; нельзя перезагрузить автономные подпрограммы.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Перезагрузка подпрограмм

Возможность *перезагрузки* (*overloading*), предоставляемая PL/SQL, позволяет разрабатывать несколько пакетных подпрограмм с одним и тем же именем. Перезагрузка полезна, когда подпрограмма принимает похожий набор параметров с другими типами данных. Например, функция `TO_CHAR` может быть вызвана по-разному для преобразования числового данного или даты в символьную строку.

PL/SQL позволяет перезагружать пакетные подпрограммы и методы типов объектов.

Основное правило состоит в том, что для подпрограмм можно использовать одинаковые имена до тех, пока их формальные параметры отличаются по количеству, порядку или семейству типов данных.

Перезагрузкой можно воспользоваться, если:

- Похожи правила обработки нескольких подпрограмм, но типы параметров или их количество разное.
- Необходимы альтернативные способы поиска данных с использованием различных критериев. Например, может понадобиться найти сотрудников по их номерам, а также по фамилиям. Внутренняя логика поиска одинаковая, но критерии разные.
- Функциональные возможности расширяются, и вы не хотите изменять существующий код.

Примечание. Автономные (stand-alone) подпрограммы не могут быть перезагружаемыми. В этом курсе не рассматривается кодирование подпрограмм, используемых в качестве методов объектных типов.

Перезагрузка подпрограмм (продолжение)

Ограничения

Нельзя *перезагрузить* (*overload*):

- две подпрограммы, если их формальные параметры отличаются только по типу данных и эти типы принадлежат одному и тому же семейству (типы данных `NUMBER` и `DECIMAL` принадлежат одному и тому же семейству);
- две подпрограммы, если их формальные параметры отличаются только по подтипу данных и эти различные подтипы основаны на типах данных одного и того же семейства (`VARCHAR` и `STRING` – это подтипы PL/SQL типа данных `VARCHAR2`);
- две функции, которые отличаются только типом возвращаемого значения, даже если типы относятся к разным семействам.

Перезарузка подпрограмм с выше перечисленными особенностями приведет к появлению ошибки в ходе выполнения.

Примечание. Выше перечисленные ограничения применимы, если имена параметров также одинаковы. При использовании различных имен параметров можно вызвать подпрограмму, задавая параметры по их именам.

Разрешение вызовов

Компилятор пытается найти декларативную информацию, соответствующую данному вызову (т.е. объявление соответствующей подпрограммы). Сначала компилятор ищет эту информацию в текущей (локальной) области видимости, а затем, если требуется, во внешних областях видимости. Поиск прекращается, когда компилятор находит одно или несколько объявлений подпрограмм, имя в которых совпадает с именем вызываемой подпрограммы. Если подпрограммы на одном и том же уровне видимости имеют одинаковые имена, компилятору требуется точное совпадение по количеству, порядку и типам данных между фактическими и формальными параметрами.

Перезагрузка: пример

```
CREATE OR REPLACE PACKAGE dept_pkg IS
  PROCEDURE add_department(deptno NUMBER,
   name VARCHAR2 := 'unknown', loc NUMBER := 1700);
  PROCEDURE add_department(
   name VARCHAR2 := 'unknown', loc NUMBER := 1700);
END dept_pkg;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Перезагрузка: пример

На слайде показана спецификация пакета `dept_pkg` с перезагружаемыми процедурами `add_department`. Первое определение включает три параметра, значения которых используются для вставки записи о новом отделе в таблицу. В соответствии со вторым определением процедура принимает только два параметра, а номер отдела задается с помощью последовательности.

Примечание

В примере для аргументов используются базовые типы данных, которые позволяют быть уверенным в том, что вставляемые значения поместятся в предоставляемом пространстве. Для задания типов данных переменных, используемых для вставки значений в столбцы таблиц базы данных, лучше использовать атрибут `%TYPE`. Пример:

```
PROCEDURE add_department
 (deptno departments.department_id%TYPE,
  name departments.department_name%TYPE := 'unknown',
  loc departments.location_id%TYPE := 1700);
```

Перезагрузка: пример

```
CREATE OR REPLACE PACKAGE BODY dept_pkg  IS
  PROCEDURE add_department (deptno NUMBER,
   name VARCHAR2:='unknown', loc NUMBER:=1700) IS
  BEGIN
    INSERT INTO departments(department_id,
      department_name, location_id)
    VALUES  (deptno, name, loc);
  END add_department;

  PROCEDURE add_department (
    name VARCHAR2:='unknown', loc NUMBER:=1700) IS
  BEGIN
    INSERT INTO departments (department_id,
      department_name, location_id)
    VALUES (departments_seq.NEXTVAL, name, loc);
  END add_department;
 END dept_pkg;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Перезагрузка: пример (продолжение)

Когда процедура `add_department` вызывается с явно заданным номером отдела, PL/SQL использует первый вариант процедуры. Пример:

```
EXECUTE dept_pkg.add_department(980,'Education',2500)
SELECT * FROM departments
WHERE department_id = 980;
```

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
980	Education		2500

Если процедура `add_department` вызывается без номера отдела, PL/SQL использует второй вариант процедуры. Пример:

```
EXECUTE dept_pkg.add_department ('Training', 2400)
SELECT * FROM departments
WHERE department_name = 'Training';
```

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
320	Training		2400

Перегрузка и пакет STANDARD

- В пакете STANDARD определена среда и встроенные функции PL/SQL.
- Большинство встроенных функций перезагружаемые. Пример функции TO_CHAR:

```
FUNCTION TO_CHAR (p1 DATE) RETURN VARCHAR2;
FUNCTION TO_CHAR (p2 NUMBER) RETURN VARCHAR2;
FUNCTION TO_CHAR (p1 DATE, P2 VARCHAR2) RETURN
VARCHAR2;
FUNCTION TO_CHAR (p1 NUMBER, P2 VARCHAR2) RETURN
VARCHAR2;
```

- Подпрограмма PL/SQL с таким же именем, как у встроенной подпрограммы, переопределяет стандартное объявление в локальном контексте, если только перед именем подпрограммы не указано имя пакета в качестве квалификатора.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Перегрузка и пакет STANDARD

Пакет STANDARD определяет среду PL/SQL и глобально объявленные типы, исключения и подпрограммы, которые автоматически доступны программам PL/SQL. Большинство *встроенных функций* (*built-in functions*), находящихся в пакете STANDARD, перезагружаемые. Например, функция TO_CHAR имеет четыре различных объявления, как показано на слайде. Функция может принимать значение типа DATE или NUMBER и преобразовывать его в символьный тип данных. В вызове функции также может задаваться формат, в который преобразуется дата или число.

Если стандартная подпрограмма повторно объявляется в другой программе PL/SQL, такое локальное объявление подменяет стандартную или встроенную подпрограмму. Чтобы обратиться к стандартной подпрограмме, необходимо перед ней указать имя пакета в качестве квалификатора. Например, если функция TO_CHAR переопределена, к встроенной функции можно обратиться следующим образом: STANDARD.TO_CHAR.

Когда стандартная подпрограмма повторно объявляется как автономная подпрограмма, для обращения к ней необходимо указать имя схемы в качестве квалификатора перед именем подпрограммы, например, SCOTT.TO_CHAR.

Использование предварительного объявления

- В языках с блочной структурой (например, в PL/SQL) идентификаторы должны быть объявлены прежде, чем на них можно будет ссылаться.
- Пример проблемы, связанной с ссылкой:

```
CREATE OR REPLACE PACKAGE BODY forward_pkg IS
  PROCEDURE award_bonus(. . .) IS
  BEGIN

    calc_rating (. . .);    --неверная ссылка

  END;

  PROCEDURE calc_rating (. . .) IS
  BEGIN
    ...
  END;
END forward_pkg;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование предварительного объявления

PL/SQL, как и другие языки с блочной структурой, не допускает ссылки на идентификаторы, которые еще не были объявлены. Прежде, чем использовать идентификатор, вы должны его объявить. Например, подпрограмма должна быть объявлена прежде, чем ее можно будет вызывать.

Используемые стандарты кодирования часто требуют, чтобы подпрограммы в пакете располагались в алфавитном порядке для упрощения их поиска. В таком случае может возникнуть проблема, приведенная на слайде. Процедуру `calc_rating` нельзя вызвать, поскольку она еще не была объявлена.

В данном случае проблему, вызванную использованием запрещенной ссылки, можно преодолеть, поменяв порядок расположения двух процедур на противоположный. Но такое простое решение не всегда подходит, если правила кодирования требуют располагать подпрограммы в алфавитном порядке.

Решение такой проблемы заключается в использовании *предварительного объявления* (*forward declaration*), предоставляемого PL/SQL. Эта форма позволяет объявить заголовок подпрограммы (спецификацию подпрограммы, которая заканчивается точкой с запятой).

Примечание. Ошибка компиляции, показывающая, что процедура `calc_rating` не определена, может быть вызвана тем, что `CALC_RATING` – это частная процедура пакета. Когда `calc_rating` объявлена в спецификации пакета, ссылка на общедоступную процедуру разрешается компилятором.

Использование предварительного объявления

В теле пакета предварительное объявление – это спецификация частной подпрограммы, заканчивающаяся точкой с запятой.

```
CREATE OR REPLACE PACKAGE BODY forward_pkg IS
 PROCEDURE calc_rating (...);-- предварительное
                             -- объявление
 -- Подпрограммы определяются в алфавитном порядке

 PROCEDURE award_bonus(...) IS
 BEGIN
   calc_rating (...);        -- ссылка разрешена!
   . . .
 END;

 PROCEDURE calc_rating (...) IS -- реализация
 BEGIN
   . . .
 END;
END forward_pkg;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование предварительного объявления (продолжение)

Как было отмечено ранее, PL/SQL предоставляет специальную возможность для объявления подпрограмм, называемую предварительным объявлением. Такое объявление может потребоваться в теле пакета для частной подпрограммы. Оно содержит спецификацию подпрограммы, заканчивающуюся точкой с запятой.

Предварительное объявление помогает:

- определить подпрограмм в алфавитном или логическом порядке;
- определить взаимно рекурсивных подпрограмм; взаимно рекурсивные программы – это программы, которые вызывают друг друга прямо или косвенно.
- объединить подпрограммы в пакет.

При использовании предварительного объявления:

- список формальных параметров должен присутствовать как в предварительном объявлении, так и в теле подпрограммы.
- тело подпрограммы может появиться в любом месте после предварительного объявления, но и тело, и это объявление должны быть в пределах одной и той же программной единицы.

Предварительные объявления и пакеты

Обычно предварительные объявления позволяют объединять взаимосвязанные подпрограммы в пакеты. Спецификации подпрограмм становятся частью спецификации пакета, а тела подпрограмм – частью тела пакета. Для общедоступных подпрограмм, объявленных в спецификации, не требуется предварительного объявления.

Блок инициализации пакета

Блок в конце тела пакета выполняется один раз и используется для инициализации общедоступных и частных переменных.

```
CREATE OR REPLACE PACKAGE taxes IS
  tax   NUMBER;
  ...  -- объявление всех общедоступных процедур/функций
END taxes;
/
CREATE OR REPLACE PACKAGE BODY taxes IS
  ...--объявление всех частных переменных
  ...--определение общедоступных/частных процедур/функций
 BEGIN
   SELECT   rate_value INTO tax
   FROM     tax_rates
   WHERE    rate_name = 'TAX';
END taxes;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Блок инициализации пакета

При первом обращении к компоненте пакета весь пакет загружается в память пользовательского сеанса. По умолчанию переменные инициализируются как неопределенные (`NULL`), если они явно не проинициализированы. Чтобы переменные получили начальные значения, можно:

- использовать операторы присваивания в объявлении переменных, когда инициализация может быть выполнена таким простым образом;
- добавить блок кода в конце тела пакета, если инициализация должна быть произведена более сложным образом.

Блок кода в конце тела пакета – это *блок инициализации пакета* (*package initialization block*), который выполняется только один раз при первом вызове пакета в сеансе пользователя.

В примере на слайде общедоступная переменная `tax` инициализируется на основе значения из таблицы `tax_rates` при первом обращении к пакету `taxes`.

Примечание. Код блока инициализации переопределяет значение переменной, проинициализированной при объявлении с помощью оператора присваивания. Конец блока инициализации задается ключевым словом `END`, которое завершает тело пакета.

Ограничения на пакетные функции, используемые в SQL

- **Пакетные функции можно использовать в командах SQL.**
- **Ограничения:**
  - **Функция, вызываемая из запроса или команд DML, не может завершать текущую транзакцию, создавать или выполнять откат к точке сохранения, а также выполнять команды `ALTER SYSTEM` и `ALTER SESSION`.**
  - **Функция, вызываемая из команды запроса или команды DML в параллельном режиме, не может выполнять команду DML или модифицировать базу данных.**
  - **Функция, вызываемая из команды DML, не может читать или модифицировать таблицу, которая модифицируется этой командой DML.**

Примечание: вызовы из функции подпрограмм, нарушающих указанные ограничения, не разрешаются.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Ограничения на пакетные функции, используемые в SQL

Чтобы сервер Oracle мог выполнить команду SQL, которая вызывает хранимую функцию, он должен знать *уровень чистоты* (*purity level*) этой хранимой функции, т.е. свободна ли она от побочных эффектов, приведенных на слайде. В общем случае побочные эффекты – это изменения в таблицах базы данных или общедоступных пакетных переменных (объявленных в спецификации пакета). Побочные эффекты могут замедлить выполнение запроса, привести к появлению результатов, зависимых от последовательности выполнения подпрограмм (и, следовательно, непредсказуемых), а также потребовать того, чтобы переменные, в которых хранится *состояние пакета* (*package state*), сохранялись от сеанса к сеансу. Различные побочные эффекты не разрешены, когда функция вызывается из SQL-запроса или команды DML. Поэтому к хранимым функциям, вызываемым из SQL-выражений, применяются следующие ограничения.

- Функция, вызываемая из запроса или команды DML, не может завершать текущую транзакцию, создавать и выполнять откат к точке сохранения, а также выполнять команды `ALTER SYSTEM` и `ALTER SESSION`.
- Функция, вызываемая из команды запроса или параллельно выполняемой команды DML, не может выполнять команду DML или иным способом вносить изменения в базу данных.
- Функция, вызываемая из команды DML, не может читать или обновлять конкретную таблицу, изменяемую этой командой DML.

Примечание. В версиях до Oracle8*i* проверка чистота выполнялась во время компиляции с помощью директивы компилятору `PRAGMA RESTRICT_REFERENCES`, задаваемой в спецификации пакета. Начиная с Oracle8*i*, чистота функции проверяется во время выполнения.

Пример пакетной функции, используемой в SQL

```
CREATE OR REPLACE PACKAGE taxes_pkg IS
  FUNCTION tax (value IN NUMBER) RETURN NUMBER;
END taxes_pkg;
/
CREATE OR REPLACE PACKAGE BODY taxes_pkg IS
  FUNCTION tax (value IN NUMBER) RETURN NUMBER IS
    rate NUMBER := 0.08;
  BEGIN
    RETURN (value * rate);
  END tax;
END taxes_pkg;
/
```

```
SELECT taxes_pkg.tax(salary), salary, last_name
FROM   employees;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Пример пакетной функции, используемой в SQL

В первом примере показывается создание спецификации и тела пакета `taxes_pkg`, в который инкапсулируется функция `tax`. Во втором примере пакетная функция `tax` вызывается в команде `SELECT`. Ниже приведены выходные результаты:

TAXES_PACK.TAX(SALARY)	SALARY	LAST_NAME
1920	24000	King
1360	17000	Kochhar
1360	17000	De Haan
720	9000	Hunold
480	6000	Ernst
422.4	5280	Austin
422.4	5280	Pataballa
369.6	4620	Lorentz
960	12000	Greenberg

...

109 rows selected.

П

В версиях до Oracle8i необходимо проверить уровень чистоты функции с помощью задаваемой в спецификации пакета директивы `PRAGMA RESTRICT_REFERENCES`. Если эта директива не задана, выдается сообщение об ошибке. В нем говорится, что при вызове пакетной функции `TAX` из запроса не гарантируется отсутствие команд, изменяющих информацию базы данных.

Устойчивое состояние пакетов

Совокупность пакетных переменных и значений определяет состояние пакета.

- **Состояние пакета инициализируется при первой загрузке пакета**
- **Состояние пакета устойчивое (по умолчанию) в течение времени жизни сеанса**
  - **Сохраняется в глобальной области пользователя (User Global Area, UGA)**
  - **Уникальное для каждого сеанса**
  - **Может изменяться при вызове пакетных подпрограмм и общедоступных переменных**
- **Состояние пакета неустойчивое в течение времени жизни сеанса, если в спецификации пакета указывается `PRAGMA SERIALLY_REUSABLE`**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Устойчивое состояние пакетов

Совокупность общедоступных и частных пакетных переменных отражает *состояние пакета* (*package state*) в сеансе пользователя. Таким образом, состояние пакета – это набор значений, хранимых во всех переменных пакета в данный момент времени. Обычно состояние пакета существует в течение времени жизни сеанса пользователя.

Пакетные переменные инициализируются при первой загрузке пакета в память в сеансе пользователя. Пакетные переменные по умолчанию уникальны для каждого сеанса и содержат свои значения до прекращения пользовательского сеанса. Другими словами, переменные хранятся в памяти UGA, выделяемой базой данных для каждого пользовательского сеанса.

Состояние пакета изменяется, когда вызывается подпрограмма пакета и она изменяет состояние переменной. Состояние общедоступного (public) пакета можно непосредственно изменить с помощью операции соответствующего типа.

Примечание. После добавления директивы `PRAGMA SERIALLY_RESUABLE` в спецификацию пакета база данных помещает пакетные переменные в *системную глобальную область* (*System Global Area*, SGA), разделяемую пользовательскими сеансами. В таком случае состояние пакета сопровождается в течение времени жизни вызываемой подпрограммы или единичного обращения к пакетной конструкции. Директива `SERIALLY_REUSABLE` полезна, если необходимо сберечь память и не требуется устойчивого состояния для каждого пользовательского сеанса.

Устойчивое состояние пакетных переменных: пример

Время	События	Состояние для: -Scott- STD	MAX	-Jones- STD	MAX
9:00	Scott> EXECUTE comm_pkg.reset_comm(0.25)	0.10 0.25	0.4	-	0.4
9:30	Jones> INSERT INTO employees(last_name,commission_pct) VALUES('Madonna', 0.8);	0.25	0.4		0.8
9:35	Jones> EXECUTE comm_pkg.reset_comm (0.5)	0.25	0.4	0.1 0.5	0.8
10:00	Scott> EXECUTE comm_pkg.reset_comm(0.6) Err -20210 'Bad Commission'	0.25	0.4	0.5	0.8
11:00	Jones> ROLLBACK;	0.25	0.4	0.5	0.4
11:01	EXIT ...	0.25	0.4	-	0.4
12:00	EXEC comm_pkg.reset_comm(0.2)	0.25	0.4	0.2	0.4

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Устойчивое состояние пакетных переменных: пример

На слайде представлена последовательность действий пользователей Scott и Jones, использующих пакет `comm_pkg` (рассмотренный в уроке “Создание пакетов”). Пользователи выполняют процедуру `reset_comm`, которая вызывает функцию `validate` для проверки нового размера комиссионных.

Последовательность шагов на слайде показывает, как поддерживается устойчивое состояние пакетной переменной `std_comm` в каждом пользовательском сеансе.

9:00: Scott вызывает процедуру `reset_comm` с новым размером комиссионных, равным 0.25. Начальное состояние пакетной переменной `std_comm` 0.10. Величина 0.25 сравнивается с максимальным значением комиссионных, равным 0.4 (полученным из таблицы `EMPLOYEES`). Т.к. 0.25<0.4, глобальная переменная становится равной 0.25.

9:30: Jones добавляет новую строку в таблицу `EMPLOYEES`, в которой доля комиссионных (`commission_pct`) равна 0.8. Изменение не фиксируется, поэтому оно видимо только пользователю Jones. На состояние сеанса пользователя Scott это изменение никакого влияние не оказывает.

9:35: Jones вызывает процедуру `reset_comm` с новым размером комиссионных, равным 0.5. Начальное состояние в сеансе пользователя Jones для переменной `std_comm` 0.10. Оно изменяется на 0.5, так как в этом сеансе максимумом 0.8.

10:00: Scott вызывает процедуру с комиссионными в размере 0.6. Это значение превышает максимальную долю комиссионных, равную 0.4 (пользователь Scott не может видеть новое значение, поскольку Jones не завершил транзакцию). В результате изменение не происходит.

11:00 – 12:00: Jones откатывает транзакцию и выходит из сеанса. При его

подсоединении в 12:00 процедура успешно выполняется и новое состояние для пакетной переменной – 0.2.

Устойчивое состояние пакетного курсора

```
CREATE OR REPLACE PACKAGE BODY curs_pkg IS
  CURSOR c IS SELECT employee_id FROM employees;
  PROCEDURE open IS
  BEGIN
    IF NOT c%ISOPEN THEN   OPEN c;  END IF;
  END open;
  FUNCTION next(n NUMBER := 1) RETURN BOOLEAN IS
    emp_id employees.employee_id%TYPE;
  BEGIN
    FOR count IN 1 .. n LOOP
      FETCH c INTO emp_id;
      EXIT WHEN c%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE('Id: ' ||(emp_id));
    END LOOP;
    RETURN c%FOUND;
  END next;
  PROCEDURE close IS BEGIN
    IF c%ISOPEN THEN  CLOSE c;  END IF;
  END close;
END curs_pkg;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Устойчивое состояние пакетного курсора

На слайде приведено тело пакета `CURS_PKG`, реализующее следующую спецификацию:

```
CREATE OR REPLACE PACKAGE curs_pkg IS
  PROCEDURE open;
  FUNCTION next(n NUMBER := 1) RETURN BOOLEAN;
  PROCEDURE close;
END curs_pkg;
```

Выполните следующие шаги, чтобы воспользоваться этим пакетом:

- Вызовите процедуру `open` для открытия курсора.
- Вызовите процедуру `next` для выборки (fetch) одной или заданного количества строк. Когда вы запрашиваете большее количество строк, чем в действительности существует, процедура обрабатывает возникающую ситуация и успешно завершается. Процедура возвращает `TRUE`, если остались необработанные строки, иначе возвращается `FALSE`.
- Вызовите процедуру `close` для закрытия курсора перед завершением или в конце обработки строк.

Примечание. Курсор объявлен внутри пакета. Поэтому на состояние такого частного (private) курсора могут оказать влияние вызываемые пакетные процедуры и функция, приведенные на слайде.

Выполнение компонентов пакета CURS_PKG

```
SET SERVEROUTPUT ON
EXECUTE curs_pkg.open
DECLARE
  more BOOLEAN := curs_pkg.next(3);
BEGIN
  IF NOT more THEN
    curs_pkg.close;
  END IF;
END;
/
RUN  -- повторное выполнение анонимного блока
EXECUTE curs_pkg.close
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Выполнение компонентов пакета CURS_PKG

Как вы помните, состояние пакетной переменной или курсора в сеансе устойчивое и передается от одной транзакции к другой. Однако состояние пакета не передается между разными сеансами одного и того же пользователя. Данные, хранящиеся в таблицах БД, характеризуются устойчивостью и используются сеансами одного и того же или разных пользователей.

Для подготовки вывода результатов в *i*SQL*Plus выполняется команда `SET SERVEROUTPUT ON`. Процедура `curs_pkg.open` открывает курсор, который остается открытым до тех пор, пока сеанс не прервется или курсор не будет явно закрыт. В декларативной секции анонимного блока выполняется функция `next`, устанавливающая начальное значение `TRUE` для переменной `more` с типом данных `BOOLEAN` (так как в таблице `employees` больше трех строк). В блоке осуществляется проверка конца результирующего набора и в случае его достижения закрытие курсора. При выполнении блока выводятся первые три строки:

```
Id :103
Id :104
Id :105
```

Команда `RUN` еще раз выполняет блок; выводятся последующие три строки:

```
Id :106
Id :107
Id :108
```

Команда `EXECUTE curs_pkg.close` закрывает пакетный курсор.

Использование таблиц PL/SQL с записями в пакетах

```
CREATE OR REPLACE PACKAGE emp_pkg IS
  TYPE emp_table_type IS TABLE OF employees%ROWTYPE
     INDEX BY BINARY_INTEGER;
  PROCEDURE get_employees(emps OUT emp_table_type);
END emp_pkg;
/
```

```
CREATE OR REPLACE PACKAGE BODY emp_pkg IS
  PROCEDURE get_employees(emps OUT emp_table_type) IS
    i BINARY_INTEGER := 0;
  BEGIN
    FOR emp_record IN (SELECT * FROM employees)
    LOOP
      emps(i) := emp_record;
      i:= i+1;
    END LOOP;
  END get_employees;
END emp_pkg;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование таблиц PL/SQL с записями в пакетах

Пакет `emp_pkg` содержит процедуру `get_employees`, которая читает строки из таблицы `EMPLOYEES` и возвращает их, используя параметр в режиме `OUT`, представляющий собой таблицу PL/SQL с записями. В примере на слайде:

- `employee_table_type` объявлен как общедоступный тип;
- `employee_table_type` используется для задания типа формального выходного параметра процедуры и переменной `employees` в вызывающем блоке (см. ниже).

В *i*SQL*Plus можно вызвать процедуру `get_employees` в анонимном блоке PL/SQL, используя переменную `employees`, как в следующем примере:

```
DECLARE
  employees  emp_pkg.emp_table_type;
BEGIN
  emp_pkg.get_employees(employees);
  DBMS_OUTPUT.PUT_LINE('Emp 4: '||employees(4).last_name);
END;
/
```

Результат выполнения:

```
Emp 4: Ernst
```

Утилита для преобразования кода PL/SQL в нечитаемый вид

- **Автономная утилита `WRAP` скрывает внутреннее содержимое приложения, преобразовывая исходный код PL/SQL в переносимый объектный код.**
- **Такое преобразование обеспечивает:**
  - **Независимость от платформы**
  - **Динамическую загрузку**
  - **Динамическое связывание**
  - **Проверку зависимостей**
  - **Стандартный импорт и экспорт, выполняемый, когда это необходимо**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Назначение утилиты `WRAP`

Автономная утилита `WRAP` преобразует исходный код PL/SQL в переносимый объектный код. Ее использование позволяет поставлять приложения PL/SQL без открытого исходного кода, который может содержать принадлежащие вам алгоритмы и структуры данных. Утилита `WRAP` конвертирует доступный для чтения исходный код в нечитаемый код. Скрывая внутреннее содержание, вы предотвращаете неправильное использование вашего приложения.

Полученный после *преобразования* (*wrap*) код (например, хранимые программные единицы PL/SQL) имеет ряд преимуществ:

- *независимость от платформы*; не требуется поставлять несколько версий одной и той же компилируемой единицы;
- допускается *динамическая загрузка*; пользователям не надо производить остановку и перелинковку для добавления новой возможности;
- разрешается *динамическое связывание*; внешние ссылки разрешаются во время загрузки;
- строгая *проверка зависимостей*; неработоспособные (со статусом `invalid`) программы автоматически перекомпилируются при вызове;
- поддержка *стандартного импорта и экспорта*; утилита экспорта/импорта может обработать файлы с кодом, полученным после преобразования утилитой `WRAP`.

Выполнение утилиты WRAP

Синтаксис командной строки:

```
WRAP INAME=имя_входного_файла [ONAME=имя_выходного_файла]
```

- **Аргумент INAME обязательный.**
- **По умолчанию расширение входного файла – .sql, если только оно не указано в имени.**
- **Аргумент ONAME необязательный.**
- **По умолчанию расширение выходного файла – .plb, если только оно не указано в имени.**

Примеры:

```
WRAP INAME=student.sql
WRAP INAME=student
WRAP INAME=student.sql ONAME=student.plb
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Выполнение утилиты WRAP

На уровне операционной системы утилита представлена исполняемым модулем WRAP. Для ее запуска введите следующую команду после приглашения ОС :

```
WRAP INAME=имя_входного_файла [ONAME=имя_выходного_файла]
```

В каждом из приведенных на слайде примеров используется входной файл student.sql и создается выходной файл student.plb.

После создания *файла с преобразованным кодом* (*wrapped file*), обычно имеющим расширение .plb, выполните его в *i*SQL*Plus, как это делается с обычными скрипт-файлами SQL. В результате компилируется и сохраняется преобразованная утилитой WRAP версия исходного кода.

Замечания

- Только аргумент INAME обязательно должен указываться. Если аргумент ONAME не задан, тогда выходному файлу присваивается такое же имя, как у входного файла, но с расширением .plb.
- Не ставьте пробелы в командной строке вокруг знаков равенства при задании аргументов INAME и ONAME.
- Входной файл может иметь любое расширение (по умолчанию – .sql).
- В определенных ОС значения аргументов INAME и ONAME могут зависеть от регистра символов.
- Обычно выходной файл значительно больше входного файла.

Результат преобразования

- **Первоначальный исходный код PL/SQL во входном файле:**

```
CREATE PACKAGE banking IS
  min_bal := 100;
  no_funds EXCEPTION;
...
END banking;
/
```

- **Преобразованный код в выходном файле:**

```
CREATE PACKAGE banking
wrapped

012abc463e ...

/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Результат преобразования

После преобразования объектный тип, пакет и подпрограмма приобретают следующий вид: заголовок, за которым следует слова `wrapped`, а затем зашифрованное тело.

Входной файл может содержать любую комбинацию команд SQL. Однако утилита `WRAP` преобразует только следующие команды `CREATE`:

- `CREATE [OR REPLACE] TYPE`
- `CREATE [OR REPLACE] TYPE BODY`
- `CREATE [OR REPLACE] PACKAGE`
- `CREATE [OR REPLACE] PACKAGE BODY`
- `CREATE [OR REPLACE] FUNCTION`
- `CREATE [OR REPLACE] PROCEDURE`

Все остальные команды `CREATE` языка SQL пропускаются в нетронутом виде в выходной файл.

Рекомендации по преобразованию кода

- **Необходимо преобразовывать только тела пакетов, но не их спецификации.**
- **Утилита `WRAP` может обнаружить синтаксические ошибки, но не в состоянии обнаружить семантические ошибки.**
- **Не следует редактировать выходной файл. Изменения вносятся в первоначальный код, который затем необходимо снова преобразовать с помощью утилиты `WRAP`.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Рекомендации по преобразованию кода

- При преобразовании пакета или объектного типа обрабатывается только тело, но не спецификация. В результате другим разработчикам предоставляется информация, необходимая для использования пакета, но при этом не раскрывается реализация.
- Когда входной файл содержит синтаксические ошибки, утилита `WRAP` сообщает о них. Однако утилита не обнаруживает семантические ошибки, так как он не может разрешить внешние ссылки. Например, не сообщается об ошибке, когда не существует таблица или представление `amp`:

```
CREATE PROCEDURE raise_salary (emp_id INTEGER, amount
NUMBER) AS
BEGIN
   UPDATE amp  -- следовало указать emp
     SET sal = sal + amount WHERE empno = emp_id;
END;
```

  Компилятор PL/SQL разрешает внешние ссылки. Поэтому о таких семантических ошибках сообщается, когда компилируется выходной файл утилиты `WRAP` (файл`.plb`).
- Выходной файл нельзя редактировать, так как его содержимое имеет нечитаемый вид. Чтобы изменить преобразованный утилитой `WRAP` объект, необходимо внести изменения в первоначальный исходный код, а затем снова обработать его утилитой `WRAP`.

Итоги

- Создание и вызов перезагружаемых подпрограмм
- Использование предварительного объявления подпрограмм
- Создание блока инициализации пакета
- Поддержка устойчивого состояния пакета
- Использование утилиты `WRAP` для преобразования кода PL/SQL

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Итоги

Перезагрузка представляет собой такую возможность, которая позволяет определять различные программы с одним и тем же именем. Иногда две подпрограммы производят одну и ту же обработку, но отличаются передаваемыми им параметрами. В этом случае логично присвоить им одно и то же имя.

PL/SQL предоставляет особую форму объявления подпрограмм, называемую предварительным объявлением. Предварительное объявление позволяет определять подпрограммы в алфавитном или логическом порядке, взаимно рекурсивные подпрограммы и объединять подпрограмм в пакет.

Блок инициализации пакета выполняется только один раз, когда пакет впервые вызывается во время пользовательского сеанса. Такую возможность можно использовать для одноразовой инициализации переменных в сеансе.

Вы можете проследить состояние пакетной переменной или курсора, которое остается неизменным в течение всего сеанса пользователя, начиная с момента первого обращения пользователя к переменной или курсору и до времени отсоединения пользователя.

Используя утилиту `WRAP`, можно получить нечитаемым исходный код PL/SQL, хранимый в базе, и защитить таким образом вашу интеллектуальную собственность.

Обзор практического занятия 4

- Использование перезагрузки подпрограмм
- Создание блока инициализации пакета
- Использование предварительного объявления
- Использование утилиты `WRAP`, препятствующей расшифровке исходного кода другими людьми

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Обзор практического занятия 4

При выполнении этого практического задания вы будете изменять существующий пакет и внесете в него перезагружаемые подпрограммы, а также будете использовать предварительное объявление.

Вы создадите блок инициализации в теле пакета для заполнения таблицы PL/SQL.

Вы будете использовать утилиту командной строки `WRAP` для того, чтобы сделать исходный код, хранимый в словаре данных, нечитаемым.

Практическое занятие 4

1. Скопируйте и измените код пакета EMP_PKG, созданный вами при решении задания 2 практического занятия 3. Сделайте процедуру ADD_EMPLOYEE перезагружаемой.
   (а). Добавьте новую процедуру ADD_EMPLOYEE с тремя параметрами: имя, фамилия и номер отдела. Сохраните и скомпилируйте измененный код.
   (б). Создайте в теле пакета код новой процедуры ADD_EMPLOYEE, в котором формируется адрес электронной почты, содержащий символы верхнего регистра и включающий первую букву имени, соединенную с первыми семи буквами фамилии. Для выполнения операции INSERT процедура должна вызывать существующую процедуру ADD_EMPLOYEE, используя ее параметры и отформатированный адрес электронной почты . Сохраните и скомпилируйте измененный код.
   (в). Вызовите новую процедуру ADD_EMPLOYEE, чтобы добавить сотрудника Samuel Joplin в отдел 30.
2. В пакете EMP_PKG создайте две перезагружаемые функции GET_EMPLOYEE.
   (а). Добавьте в спецификацию функцию GET_EMPLOYEE, принимающую параметр emp_id с типом, заданным атрибутом employees.employee_id%TYPE, а также вторую функцию GET_EMPLOYEE, принимающую параметр family_name с типом employees.last_name%TYPE. Обе функции возвращают значение с типом EMPLOYEES%ROWTYPE. Сохраните и скомпилируйте измененный код.
   (б). В теле пакета создайте код первой функции GET_EMPLOYEE для запроса данных о сотруднике по его номеру и код второй функции, в котором в условии выбора с помощью оператора равенства проверяется совпадение фамилии и параметра family_name. Сохраните и скомпилируйте измененный код тела пакета.
   (в). Добавьте в пакет вспомогательную процедуру PRINT_EMPLOYEE, принимающую значение типа EMPLOYEES%ROWTYPE в качестве параметра и выводящую на одной строке поля department_id, employee_id, first_name, last_name, job_id и salary, относящиеся к одному сотруднику. Используйте пакет DBMS_OUTPUT. Сохраните и скомпилируйте измененный код.
   (г). Используйте анонимный блок для вызова функции EMP_PKG.GET_EMPLOYEE с номером служащего 100 и фамилией 'Joplin'. Для вывода возвращаемой строки используйте процедуру PRINT_EMPLOYEE.
3. Данные об отделах изменяются редко. Поэтому производительность пакета EMP_PKG повысится после добавления общедоступной процедуры INIT_DEPARTMENTS, предназначенной для заполнения частной таблицы PL/SQL, содержащей номера отделов. Измените функцию VALID_DEPTID, так чтобы для проверки правильности значения номера отдела использовалась эта заполненная частная таблица PL/SQL.
   (а). В спецификацию пакета добавьте процедуру INIT_DEPARTMENTS без параметров.

Практическое занятие 4 (продолжение)

(б). Добавьте в тело пакета процедуру `INIT_DEPARTMENTS`, которая должна сохранить все номера отделов в частной `INDEX BY` таблице PL/SQL с именем `valid_departments`. Используйте значение столбца `department_id` в качестве индекса, объявите как `BOOLEAN` тип столбца таблицы PL/SQL и занесите в него значение `TRUE`. Объявите переменную `valid_departments` с типом `boolean_tabtype` перед всеми процедурами в коде тела пакета.

(в). Добавьте в тело пакета блок инициализации, в котором вызывается процедура `INIT_DEPARTMENTS`. Сохраните и скомпилируйте измененный код.

4. Измените процедуру проверки `VALID_DEPTID` так, чтобы в ней использовалась частная таблица PL/SQL, содержащая номера отделов.

   (а). Измените процедуру `VALID_DEPTID` так, чтобы в ней для проверки использовалась частная таблица PL/SQL, содержащая номера отделов. Сохраните и скомпилируйте измененный код.

   (б). Протестируйте код, вызвав процедуру `ADD_EMPLOYEE`, используя имя и фамилию `James Bond` и отдел 15. Что произошло ?

   (в). Вставьте новый отдел 15, назовите его `Security` и зафиксируйте изменения.

   (г). Протестируйте снова код, вызвав процедуру `ADD_EMPLOYEE`, используя имя и фамилию `James Bond` и отдел 15. Что произошло?

   (д). Выполните процедуру `EMP_PKG.INIT_DEPARTMENTS`, чтобы обновить в соответствие с последними данными об отделах внутреннюю таблицу PL/SQL.

   (е). Протестируйте код, вызвав процедуру `ADD_EMPLOYEE`, используя имя и фамилию `James Bond` и отдел 15. Что произошло?

   (ж). Удалите сотрудника `James Bond` и отдел 15 из соответствующих таблиц, зафиксируйте изменения и выполните процедуру `EMP_PKG.INIT_DEPARTMENTS`, чтобы обновить информацию об отделах.

5. Реорганизуйте спецификацию пакета и расположите в ней подпрограммы в алфавитном порядке.

   (а). Отредактируйте спецификацию пакета и расположите в ней подпрограммы по алфавиту. В *i*SQL*Plus загрузите и скомпилируйте спецификацию. Что произошло?

   (б). Отредактируйте тело пакета и расположите в нем подпрограммы по алфавиту. В *i*SQL*Plus загрузите и скомпилируйте тело пакета. Что произошло?

   (в). Исправьте ошибки компиляции, используя в теле пакета предварительное объявление, разрешающее ссылку на подпрограмму. Загрузите и пересоздайте тело пакета. Что произошло?

Если осталось время, выполните следующее упражнение.

6. Преобразуйте код тела пакета `EMP_PKG` в помощью утилиты `WRAP` и пересоздайте его.

   (а). Выполните запрос к словарю, чтобы просмотреть исходный код тела пакета `EMP_PKG`.

Практическое занятие 4 (продолжение)

(б). Запустите командное окно и выполните утилиту командной строки `WRAP` для преобразования тела пакета `EMP_PKG`. Присвойте выходному файлу имя с расширением `.plb`.

Подсказка: скопируйте файл, содержащий тело пакета (сохраненный при выполнении задания 5в), в файл `emp_pkb_b.sql`.

(в). Используя *i*SQL*Plus, загрузите и выполните файл с расширением `.plb`, содержащий преобразованный код .

(г). Выполните запрос к словарю, чтобы снова просмотреть исходный код тела пакета `EMP_PKG`. Можно понять строки выводимого кода?